

Использование языка программирования Julia для решения математических задач

Часть II

Авторы:

Чуракова Юлия Романовна

*Студентка факультета математики и компьютерных наук
Кубанского Государственного Университета*

Алексеев Евгений Ростиславович

*Кандидат технических наук, доцент
Доцент кафедры информационных технологий
Кубанского Государственного Университета*

Полная версия пособия доступна в репозитории:

<https://github.com/JuliaChurakova/Julia>

Литература

1. Алексеев, Е. Р. Scilab: Решение инженерных и математических задач: учеб. издание. / Е.Р. Алексеев, К. В. Дога, О. В. Чеснокова М. – Базальт СПО; ДМК Пресс, 2024. – 440 с.
2. Балакин, А. А. Численные методы и математическое моделирование: Учебное пособие / А.А. Балакин. – Долгопрудный: Издательский Дом «Интеллект», 2022. – 288 с.
3. Деза, Е. И. Введение в криптографию: Теоретико-числовые основы защиты информации. Учебное пособие / Е. И. Деза, Л. В. Котова. – Москва: ЛЕНАНД, 2022. – 376 с.
4. Кнауб, Л. В. Теоретико-численные методы в криптографии / Л. В. Кнауб, Е. А. Новиков, Ю. А. Шитов; Сибирский федеральный университет – Красноярск: 2011. – 160 с.

Глава 13 Решение задач линейной алгебры

13.1 Определения и свойства матриц

Матрицей размера $m \times n$ называется прямоугольная таблица элементов, состоящая из m строк и n столбцов:

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

Квадратной называется матрица, у которой число строк равно числу столбцов ($m = n$).

Главной диагональю квадратной матрицы называется совокупность элементов, у которых номер строки совпадает с номером столбца:

$$a_{11}, a_{22}, a_{33}, \dots, a_{nn}$$

Побочной диагональю называется совокупность элементов, у которых сумма индексов равна $n + 1$:

$$a_{1n}, a_{2(n-1)}, a_{3(n-2)}, \dots, a_{n1}$$

Диагональной называется квадратная матрица, у которой все элементы вне главной диагонали равны нулю:

$$D = \begin{pmatrix} d_{11} & 0 & \cdots & 0 \\ 0 & d_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & d_{nn} \end{pmatrix}$$

Треугольной называется квадратная матрица, у которой все элементы ниже (*верхняя треугольная*) или выше (*нижняя треугольная*) главной диагонали равны нулю:

$$U = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & u_{nn} \end{pmatrix}, \quad L = \begin{pmatrix} l_{11} & 0 & \cdots & 0 \\ l_{21} & l_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & \cdots & l_{nn} \end{pmatrix}$$

Транспонированной матрицей A^T называется матрица, полученная из исходной заменой строк на столбцы:

$$(A^T)_{ij} = A_{ji}$$

Симметричная матрица равна транспонированной:

$$A = A^T.$$

Определителем (детерминантом) квадратной матрицы A называется скалярная величина, обозначаемая $\det A$ или $|A|$, которая вычисляется по формуле:

$$\det A = \sum_{(\sigma_1, \sigma_2, \dots, \sigma_n)} (-1)^s a_{1\sigma_1} a_{2\sigma_2} \cdots a_{n\sigma_n},$$

где суммирование ведётся по всем перестановкам $(\sigma_1, \sigma_2, \dots, \sigma_n)$ чисел $1, 2, \dots, n$, а s - число инверсий в перестановке.

Для матрицы 2×2 :

$$\det A = \begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} = a_{11}a_{22} - a_{12}a_{21}$$

Обратной матрицей A^{-1} для квадратной матрицы A называется такая матрица, что:

$$A \cdot A^{-1} = A^{-1} \cdot A = I,$$

где I - единичная матрица. Обратная матрица существует только если $\det A \neq 0$.

Знание обратной матрицы позволяет найти решение системы линейных уравнений:

$$A \cdot x = b \quad \text{или} \quad \sum_{j=1}^n a_{ij}x_j = b_i \quad \Rightarrow \quad x = A^{-1}b.$$

Собственные числа λ и собственные векторы e матрицы A определяются как:

$$A \cdot e = \lambda \cdot e.$$

Зная собственное число λ , можно найти собственный вектор из уравнения $(A - \lambda I)e = 0$. Для треугольной матрицы ($a_{ij} = 0$ при $j < i$) собственные числа равны $\lambda_i = a_{ii}$.

13.2 Метод Крамера и метод исключения переменных

Рассмотрим систему линейных уравнений общего вида:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2 \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n = b_n \end{cases}$$

Матричная форма записи:

$$A\mathbf{x} = \mathbf{b},$$

где A – матрица коэффициентов $n \times n$, $\mathbf{x}$ – вектор неизвестных, $\mathbf{b}$ – вектор правых частей.

13.2.1 Метод Крамера

Решение системы выражается через определители:

$$x_i = \frac{\det A_i}{\det A}, \quad i = 1, 2, \dots, n$$

где A_i – матрица, полученная заменой i -го столбца матрицы A на вектор $\mathbf{b}$:

$$A_i = \begin{pmatrix} a_{11} & \dots & a_{1(i-1)} & b_1 & a_{1(i+1)} & \dots & a_{1n} \\ a_{21} & \dots & a_{2(i-1)} & b_2 & a_{2(i+1)} & \dots & a_{2n} \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & \dots & a_{n(i-1)} & b_n & a_{n(i+1)} & \dots & a_{nn} \end{pmatrix}$$

Несмотря на простоту, метод становится непрактичным для систем большой размерности, поскольку требует вычисления большого количества определителей, что приводит к росту вычислительной сложности.

13.2.2 Метод исключения переменных (метод Гаусса)

Первым шагом алгоритма является формирование расширенной матрицы $(A|\mathbf{b})$, которая объединяет матрицу коэффициентов A и вектор правых частей $\mathbf{b}$:

$$(A|\mathbf{b}) = \left(\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} & b_n \end{array} \right)$$

Первый этап решения системы уравнений, называемый прямым ходом метода Гаусса, заключается в приведении расширенной матрицы к треугольному виду.

Сначала необходимо обнулить элементы первого столбца расширенной матрицы. Для этого необходимо из каждой строки (начиная со второй) вычесть первую строку, умноженную на некоторый множитель M .

Для второй строки множитель вычисляется по формуле:

$$M = \frac{a_{21}}{a_{11}} \quad (1)$$

Для третьей строки:

$$M = \frac{a_{31}}{a_{11}} \quad (2)$$

Для i -й строки:

$$M = \frac{a_{i1}}{a_{11}} \quad (3)$$

После вычисления множителя выполняется преобразование строки:

$$\begin{aligned} a_{ij} &= a_{ij} - M \cdot a_{1j}, & j &= 1, 2, 3, \dots, n \\ b_i &= b_i - M \cdot b_1 \end{aligned} \quad (4)$$

После завершения прямого хода система принимает верхнетреугольный вид:

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ 0 & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

Обратный ход метода Гаусса выполняется снизу вверх:

$$x_n = \frac{b_n}{a_{nn}}$$

В случае, если $a_{nn} = 0$ и $b_n = 0$, то система имеет бесконечное множество решений. Если $a_{nn} = 0$ и $b_n \neq 0$, то система решения не имеет.

$$x_{n-1} = \frac{b_{n-1} - a_{n-1,n}x_n}{a_{n-1,n-1}}$$

$$x_{n-2} = \frac{b_{n-2} - a_{n-2,n-1}x_{n-1} - a_{n-2,n}x_n}{a_{n-2,n-2}}$$

Общая формула для k -й переменной:

$$x_k = \frac{b_k - \sum_{j=k+1}^n a_{kj}x_j}{a_{kk}}, \quad k = n, n-1, \dots, 1$$

При численной реализации метода Гаусса возникают проблемы, когда диагональные элементы a_{nn} оказываются малыми по величине. В таких случаях деление на малые числа приводит к значительным ошибкам округления и снижает точность вычислений.

Избежать этого можно, выбрав главный (максимальный по модулю) элемент среди столбца a_{in} , и перестановкой строк переведя его на главную диагональ перед исключением. Такая процедура заметно уменьшит погрешность при численной реализации метода Гаусса. Этот метод называют **методом Гаусса с выбором главного элемента**.

Задача 1. Написать программу решения системы линейных алгебраических уравнений методом Гаусса на языке Julia. Метод Гаусса оформить в виде функции.

```
In [26]: function gauge(A, B, n)
           a = copy(A)
           b = copy(B)

           for k in 1:n
               maxindex = k - 1 + argmax(abs.(a[k:end, k]))
               #Поиск строки с максимальным по модулю элементом в текущем столбце k
               # abs.(a[k:end, k]) - берём модуль элементов текущего столбца, начин
```

```

# argmax() - возвращает индекс максимального значения в этой строке
# т.к. argmax возвращает индекс относительно подмассива (начиная с 1)
# нужно прибавить (k - 1), чтобы получить "глобальный" индекс в исходной матрице
a[k, :], a[maxindex, :] = a[maxindex, :], a[k, :] # меняем местами k-ю строку с строкой, имеющей максимальное значение
b[k], b[maxindex] = b[maxindex], b[k] # также переставляем соответствующие элементы вектора b
for i in k+1:n
    M = a[i, k] / a[k, k]
    a[i, k:n] .-= M .* a[k, k:n]
    b[i] -= M * b[k]
end
end

x = zeros(n)
for i in n:-1:1
    x[i] = (b[i] - sum(a[i, i+1:end] .* x[i+1:end])) / a[i, i]
end

return x
end

println("Введите количество строк (n)")
n = parse{Int, readline()}
println("Введенное значение n = ", n)
println("Введите количество столбцов (m):")
m = parse{Int, readline()}
println("Введенное значение m = ", m)

println("Введите элементы матрицы A($n x $m)")
A = zeros{Float64, n, m}
for i in 1:n
    for j in 1:m
        t = parse{Float64, readline()}
        A[i,j] = t
    end
end

println("Введите элементы массива b($n)")
a = zeros{Float64, n}
for i in 1:n
    k = parse{Float64, readline()}
    a[i] = k
end

println("\nВведенная матрица A:")
for i in 1:n
    println(A[i, :])
end

println("\nВведенный вектор b:")
println(a)

coef = gauge(A, a,n)

```

Введите количество строк (n)

Введенное значение $n = 2$
Введите количество столбцов (m):
Введенное значение $m = 2$
Введите элементы матрицы $A(2 \times 2)$
Введите элементы массива $b(2)$
Введенная матрица A :
[3.0, 4.0]
[6.0, 7.0]

Введенный вектор b :
[8.0, 4.0]

Out[26]: 2-element Vector{Float64}:
-13.333333333333334
12.0

13.3 Итерационные методы

Итерационные методы лучше применять в случае, когда мы имеем дело с задачами большой размерности, где метод Гаусса становится вычислительно неэффективным.

13.3.1 Метод простой итерации

Предположим, что некоторая матрица B имеет обратную матрицу, то есть её определитель отличен от нуля: $\det B \neq 0$. Тогда исходное уравнение

$$Ax = b$$

можно преобразовать к эквивалентному виду:

$$x = x + B(b - Ax)$$

Данное представление позволяет построить итерационный процесс:

$$x^{(k+1)} = x^{(k)} + B(b - Ax^{(k)}) = Dx^{(k)} + c,$$

где $D = I - BA$, а $c = Bb$.

Итерационный процесс сходится к точному решению при любом начальном приближении тогда и только тогда, когда все собственные значения λ_i матрицы D меньше 1 по модулю:

$$|\lambda_i(D)| < 1 \quad \text{для всех } i = 1, 2, \dots, n$$

Матрицу A можно представить в виде суммы трех матриц:

$$A = D + L + U$$

, где:

- D – диагональная часть матрицы A ;

- L - строго нижняя треугольная часть;
- U - строго верхняя треугольная часть.

$$D = \begin{pmatrix} a_{11} & 0 & \cdots & 0 \\ 0 & a_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn} \end{pmatrix}, \quad L = \begin{pmatrix} 0 & 0 & \cdots & 0 \\ a_{21} & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & 0 \end{pmatrix}, \quad U = \begin{pmatrix} 0 & a_1 & & \\ 0 & 0 & & \\ \vdots & \vdots & \ddots & \\ 0 & 0 & \cdots & 0 \end{pmatrix}$$

13.3.2 Метод Якоби

Представим матрицу A в виде суммы $A = D + L + U$ и выбираем $B = D^{-1}$.

Тогда:

$$x^{(k+1)} = x^{(k)} + D^{-1}(b - (D + L + U)x^{(k)})$$

Упрощаем:

$$x^{(k+1)} = D^{-1}(b - (L + U)x^{(k)})$$

Явная формула для вычисления компонент решения на каждом шаге:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i, i=1}^n a_{ij} x_j^{(k)} \right)$$

Достаточный признак сходимости метода Якоби.

В случае диагонального преобладания в матрице A метод Якоби сходится

$$|a_{ii}| > \sum_{j \neq i, i=1}^n |a_{ij}|$$

13.3.3 Метод Гаусса-Зейделя

Представим матрицу A в виде суммы нижней треугольной матрицы L и строго верхней треугольной матрицы U :

$$A = L + U$$

$$L = \begin{pmatrix} a_{11} & 0 & \cdots & 0 \\ a_{21} & a_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix}, \quad U = \begin{pmatrix} 0 & a_{12} & \cdots & a_{1n} \\ 0 & 0 & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{pmatrix}$$

Итерационный процесс:

$$\mathbf{x}^{(k+1)} = L^{-1}(\mathbf{b} - U\mathbf{x}^{(k)})$$

Явная формула для вычисления компонент решения на каждом шаге:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n$$

Достаточный признак сходимости метода Якоби и метода Гаусса-Зейделя.

В случае диагонального преобладания в матрице A метод Якоби и метод Гаусса-Зейделя сходятся.

$$|a_{ii}| > \sum_{j \neq i, i=1}^n |a_{ij}|$$

Критерии остановки:

$$\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| < \varepsilon$$

или

$$\|A\mathbf{x}^{(k)} - \mathbf{b}\| < \varepsilon$$

```
In [14]: function jacobi(A, b; tol=1e-6, max_iter=1000)
    n = length(b)
    x = zeros(n)
    x_new = zeros(n)
    for k in 1:max_iter
        for i in 1:n
            s = sum(A[i, j] * x[j] for j in 1:n if j != i)
            x_new[i] = (b[i] - s) / A[i, i]
        end
        delta = sum(abs.(x_new .- x))
        if delta < tol
            println("Сходимость достигнута на итерации $k")
            return x_new
        end
        copyto!(x, x_new)
    end
    println("Достигнуто максимальное число итераций ($max_iter)")
    return x
end

function gauss_seidel(A, b; tol=1e-6, max_iter=1000)
    n = length(b)
    x = zeros(n)
```

```

    for k in 1:max_iter
        x_old = copy(x)
        for i in 1:n
            s1 = sum(A[i, 1:i-1] .* x[1:i-1])
            s2 = sum(A[i, i+1:n] .* x_old[i+1:n])
            x[i] = (b[i] - (s1 + s2)) / A[i, i]
        end
        delta = sum(abs.(x .- x_old))
        if delta < tol
            println("Сходимость достигнута на итерации $k")
            return x
        end
    end
    println("Достигнуто максимальное число итераций ($max_iter)")
    return x
end

function check_diagonal_dominance(A)
    n = size(A, 1)
    for i in 1:n
        diag_val = abs(A[i, i])
        row_sum = sum(abs.(A[i, :])) - diag_val
        if diag_val <= row_sum
            println("Строка $i: |a_ii| = $diag_val ≤ Σ|a_ij| = $row_sum")
            return false
        end
    end
    return true
end

A = [10.0 8.0 1.0; 1.0 8.0 2.0; 2.0 1.0 6.0]
b = [13.0, 11.0, 9.0]

print("Проверка диагонального преобладания: ")
if check_diagonal_dominance(A)
    println("матрица имеет строгое диагональное преобладание\n")
else
    println("матрица НЕ имеет строгого диагонального преобладания\n")
end
println("МЕТОД ЯКОБИ")
x_jacobi = jacobi(A, b)
println("Решение: ", round.(x_jacobi, digits=6), "\n")
println("МЕТОД ГАУССА-ЗЕЙДЕЛЯ")
x_gs = gauss_seidel(A, b)
println("Решение: ", round.(x_gs, digits=6), "\n")

```

Проверка диагонального преобладания: матрица имеет строгое диагональное преобладание

МЕТОД ЯКОБИ

Сходимость достигнута на итерации 27

Решение: [0.356643, 1.027972, 1.20979]

МЕТОД ГАУССА-ЗЕЙДЕЛЯ

Сходимость достигнута на итерации 13

Решение: [0.356643, 1.027972, 1.20979]

13.4 Библиотека для решения задач линейной алгебры

Для решения задач линейной алгебры предназначена библиотека **LinearAlgebra**.

Для получения дополнительной информации о пакете см. [документацию](#).

Некоторые из функций этой библиотеки представлены в таблице ниже:

Команда	Описание
norm(A)	Функция используется для вычисления нормы матрицы или вектора.
det(A)	Функция вычисляет определитель матрицы A.
inv(A)	Функция используется для нахождения обратной матрицы A.
eigvals(A)	Функция возвращает собственные значения матрицы A.
eigvecs(A)	Функция возвращает собственные векторы матрицы A.
qr(A)	Функция выполняет QR-разложение матрицы A на ортогональную матрицу Q и верхнюю треугольную матрицу R.
lu(A)	Функция выполняет LU-разложение матрицы A на нижнюю треугольную матрицу L и верхнюю треугольную матрицу U.
svd(A):	Функция выполняет сингулярное разложение матрицы A на матрицы левых и правых сингулярных векторов U и V соответственно, а также диагональную матрицу сингулярных значений Σ .
dot(x, y)	Функция используется для вычисления скалярного произведения двух векторов x и y.
cross(x, y)	Функция используется для вычисления векторного произведения двух трехмерных векторов x и y.

Команда	Описание
<code>adjoint(A):</code>	Функция возвращает сопряженно-транспонированную матрицу A.
<code>rank(A)</code>	Функция возвращает ранг матрицы A.
<code>trace(A)</code>	Функция возвращает след матрицы A, т.е. сумму элементов главной диагонали.

Задача 1. Написать программу решения системы линейных алгебраических уравнений на языке Julia. Для тестирования программы решения систем линейных алгебраических уравнений будем использовать следующую тестовую систему:

$$A \cdot x = b$$

Матрица $A(N, N)$ формируется следующим образом. Диагональные элементы равны $2N$, все остальные элементы матрицы равны 1.

Вектор правых частей **b** определяется формулой

$$b_i = \frac{n \cdot (n+1)}{2} + i \cdot (2 \cdot (n - 1)), \text{ если элементы нумеруются с 1 и}$$

$$b_i = \frac{n \cdot (n+1)}{2} + (i + 1) \cdot (2 \cdot (n - 1)), \text{ если элементы нумеруются с 0.}$$

Матрица A обладает следующими особенностями:

1. Матрица A имеет диагональное преобладание, что позволяет использовать её в качестве тестовой для итерационных методов (Зейделя, Якоби и т. д.).
2. Заранее известно решение системы любой размерности $x=(1,2,3,4,5\dots)$.

```
In [3]: using LinearAlgebra
using Printf

# Ввод размера матрицы
println("Введите размер матрицы N ")
N = parse{Int, readline()}
println("N = ", N)

# Инициализация матрицы A и вектора b
A = ones(N, N)
b = ones(N)

# Заполнение матрицы A и вектора b
for i in 1:N
    for j in 1:N
        if i == j
            A[i, j] = 2*N # Диагональные элементы = 2N
        else
            A[i, j] = 1    # Остальные элементы = 1
        end
    end
end
```

```

    b[i] = (N * (N + 1)) / 2 + i * (2*(N - 1))
end

# Вывод матрицы A
println("\nМатрица A:")
for i in 1:N
    for j in 1:N
        @printf("%5.1f ", A[i, j])
    end
    println()
end

# Вывод вектора b
println("\nВектор b:")
for i in 1:N
    @printf("%5.1f\n", b[i])
end

# Решение системы
x = A \ b

# Вывод решения
println("\nКорни уравнения:")
for i in 1:N
    @printf("x[%d] = %8.4f\n", i, x[i])
end

r = A * x - b

println("\nМассив ошибок (невязка):")
for i in 1:N
    @printf("r[%d] = %.15e\n", i, r[i])
end

```

Введите размер матрицы N

N = 3

Матрица A:

6.0	1.0	1.0
1.0	6.0	1.0
1.0	1.0	6.0

Вектор b:

10.0
14.0
18.0

Корни уравнения:

x[1]	=	0.9500
x[2]	=	1.7500
x[3]	=	2.5500

Массив ошибок (невязка):

r[1]	=	0.0000000000000000e+00
r[2]	=	0.0000000000000000e+00
r[3]	=	-3.552713678800501e-15

Задача 2. Написать программу нахождения определителя матрицы, вычисления обратной матрицы, нахождения собственные значений и собственные векторы матрицы.

```
In [5]: using LinearAlgebra
using Printf

println("Введите размер матрицы N ")
N = parse{Int, readline()}
println("N = ", N)
A = ones(N, N)

for i in 1:N
    for j in 1:N
        if i == j
            A[i, j] = 2N
        else
            A[i, j] = 1
        end
    end
end

@printf("Исходная матрица A:\n");
for i in 1:N
    for j in 1:N
        @printf("%8.2f\t", A[i, j])
    end
    @printf("\n");
end

a1 = inv(A)
@printf("\nОбратная матрица к матрице A:\n");
for i in 1:N
```

```

        for j in 1:N
            @printf("%8.5f\t", a1[i, j])
        end
        @printf("\n");
    end

    @printf("\nПроверка:\n");
    a2 = A * a1
    for i in 1:N
        for j in 1:N
            @printf("%8.5f\t", a2[i, j])
        end
        @printf("\n");
    end

    dt = det(A)
    @printf("\n0пределитель:%8.4f\n", dt)

    a3 = eigvals(A)
    a4 = eigvecs(A)

    @printf("\nСобственные числа:")
    for j in 1:N
        @printf("%8.5f\t", a3[j])
    end
    @printf("\n\nСобственные вектора:\n");
    for i in 1:N
        for j in 1:N
            @printf("%8.5f\t", a4[i, j])
        end
        @printf("\n");
    end
end

```

Введите размер матрицы N

N = 4

Исходная матрица A:

8.00	1.00	1.00	1.00
1.00	8.00	1.00	1.00
1.00	1.00	8.00	1.00
1.00	1.00	1.00	8.00

Обратная матрица к матрице A:

0.12987	-0.01299	-0.01299	-0.01299
-0.01299	0.12987	-0.01299	-0.01299
-0.01299	-0.01299	0.12987	-0.01299
-0.01299	-0.01299	-0.01299	0.12987

Проверка:

1.00000	-0.00000	0.00000	0.00000
0.00000	1.00000	0.00000	0.00000
-0.00000	-0.00000	1.00000	0.00000
-0.00000	0.00000	0.00000	1.00000

Определитель: 3773.0000

Собственные числа: 7.00000 7.00000 7.00000 11.00000

Собственные вектора:

-0.40825	0.70711	-0.28868	-0.50000
-0.40825	-0.70711	-0.28868	-0.50000
0.81650	-0.00000	-0.28868	-0.50000
0.00000	0.00000	0.86603	-0.50000